VMS Defense Q&A

Comprehensive Guide: 80+ Questions for System Mastery

Section 1: System Terminology & Concepts (20 Q&A)

Q1.1: What is the fundamental difference between Vehicle Status and Vehicle Condition?

Vehicle Status and Vehicle Condition are two independent tracking systems:

- **Vehicle Status:** Operational availability. Can the vehicle be dispatched right now? States: Ready, Under Maintenance, Out of Service, Decommissioned
- **Vehicle Condition:** Physical state and readiness. Determined by the most recent Condition Check. States: Good, Needs Maintenance, Not Fit for Service

A vehicle can be Status=Ready but Condition=Needs Maintenance, or vice versa. The dashboard surfaces both independently because they answer different questions: "Can we use it?" vs "Should we use it?"

> The independence prevents a single system from masking problems. A vehicle marked "Under Maintenance" status won't be dispatched even if its last Condition Check said "Good", ensuring maintenance work doesn't get skipped.

Q1.2: What is Readiness Check and when is it performed?

A Readiness Check is a quick, time-sensitive verification that consumables and equipment are present and functional:

- **Timing:** Before dispatch (every time a vehicle is about to leave the lot)
- **Scope:** Fluid levels, tire pressure, lights, windshield wipers, emergency kit, first aid kit, radio/comms, fuel, etc.
- **Duration:** Results expire after 7 days. A vehicle ready on Monday may fail readiness by Friday if consumables deplete
- **Actor:** Custodian performs and signs off. Decision outcome: Ready or Not Ready

> *Staff is about to dispatch a vehicle for a 10-day mission. The Readiness Check from 5 days ago expired (>7 days), so a fresh check is required before the dispatch can proceed.*

Q1.3: What is Condition Check and how does it differ from Readiness Check?

A Condition Check is a thorough mechanical and structural assessment:

- **Scope:** Engine condition, suspension, brakes, body integrity, rust/corrosion, electrical systems, etc.
- **Duration:** Permanent until conditions change (not time-expiring like Readiness). A vehicle marked "Good" stays Good unless a defect is discovered
- **Actor:** Custodian or Maintenance Personnel (whoever is qualified) performs inspection
- **Outcome:** Good, Needs Maintenance, or Not Fit for Service
- **Triggers:** At vehicle intake, after major repairs, periodically per maintenance schedule, or after an incident

> Why not time-expire Condition Checks? Because replacing an engine takes months, and marking "Not Fit" doesn't change the fact. What matters is whether something *changed*, not how much *time* passed. Readiness expires because consumables run down on their own; condition doesn't unless something breaks.

Q1.4: What is an Issue Report and when is it created?

An Issue Report documents a specific defect or problem discovered on a vehicle:

- **Created by:** Any staff member who discovers a problem (during Readiness Check, Condition Check, normal operation, or incident)
- **Contents:** Description, location, severity, vehicle ID, date discovered
- **Purpose:** Formal record that a problem exists and needs attention
- **Outcomes:** May or may not result in a Maintenance Ticket. A false alarm or cosmetic issue might be logged but not actioned

Issue Reports are the *complaint*, not the *work order*. They capture what's wrong. A Maintenance Ticket is the *action plan* for fixing it.

> *A Custodian notices a slow oil leak during Readiness Check. They create an Issue Report. Later, Maintenance reviews it and decides: (a) create a ticket to fix it, (b) monitor it without immediate repair, or (c) defer it if the vehicle is needed urgently.*

Q1.5: What is a Maintenance Ticket and what are its states?

A Maintenance Ticket is a work order for repairing a vehicle. It has a clear state machine:

- **Open:** Ticket created, awaiting assignment to a mechanic
- **Active:** Mechanic is assigned and working on the ticket
- **Closed:** Work is complete (either Done with all work finished, or Decision Close if work is incomplete but intentional)
- **Cancelled:** Work was abandoned or no longer needed

A ticket can only progress Open → Active → Closed. It cannot go backward. Once closed (for any reason), a new ticket must be created if more work is needed.

> Tickets don't auto-expire or auto-close. A ticket can remain Open for months if no mechanic is available. This ensures work is never accidentally forgotten—it sits visible until someone acts on it.

Q1.6: What are Sub-Issues and what is their state machine?

A Sub-Issue is a component of work within a Maintenance Ticket. One ticket can have multiple sub-issues (one per part to replace, one per system to diagnose, etc.):

- **Open:** Sub-issue created, awaiting work
- **Under Repair:** Mechanic is actively working on it
- **For Inspection:** Work is complete, awaiting Custodian or Admin inspection
- **For Confirmation:** Inspection complete, awaiting final Admin approval
- **Done:** Work verified and approved, sub-issue closed
- **Deferred:** Work postponed (creates automatic Issue Report breadcrumb for future follow-up)

Sub-issues allow granular tracking of multi-part repairs. If a ticket has 5 sub-issues, 3 might be Done while 2 are still Under Repair.

> *A transmission overhaul ticket has 3 sub-issues: (1) Replace gaskets (Done), (2) Replace seals (Under Repair), (3) Pressure test (not yet started). The ticket remains Active until all sub-issues are Done or Deferred.*

Q1.7: What is the Main Issue and how does it relate to Sub-Issues?

The Main Issue is the core problem a Maintenance Ticket was created to address. It gets assigned to a Custodian for Tier-1 verification before moving to sub-issue work:

- **Flow:** Issue Report → Ticket created → Custodian verifies the Main Issue → If verified, mechanic creates and works on Sub-Issues
- **Purpose:** Ensures someone with vehicle knowledge confirms the problem is real before expensive diagnostic/repair work begins
- **Outcome:** Verified or Rejected. If Rejected, the ticket is closed immediately

> The Main Issue is a gate. It prevents mechanics from working on false alarms or misdiagnosed problems. By requiring Custodian sign-off before sub-issue work starts, the system avoids wasted labor on non-existent issues.

Q1.8: What are the three RBAC roles and what are their overlapping permissions?

Admin: Full system access. Creates users, manages reports, confirms tickets, archives issues.

Custodian: Vehicle lifecycle owner. Performs Readiness/Condition Checks, creates Issue Reports, verifies Main Issues, inspects sub-issue repairs.

Maintenance Personnel: Repair executor. Assigned to tickets, updates sub-issue status, completes repairs.

Overlapping permissions:

- Custodians can also perform some Admin functions (e.g., view reports)
- Maintenance Personnel can view tickets and their own assignments
- Admins can do everything Custodians can do (create issues, inspect, etc.)

Separation of duties ensures no single role owns the entire workflow. A Custodian verifies the problem; an Admin approves the solution.

Q1.9: What does "Tier-1 Verification" mean?

Tier-1 Verification is the step where a Custodian (or Admin acting as Custodian) reviews and confirms the Main Issue:

- **Gate:** A ticket cannot proceed to sub-issue work until the Main Issue is verified
- **Actor:** Custodian assigned to the ticket (not the mechanic)
- **Outcomes:** Verified (proceeds to sub-issue work) or Rejected (ticket closes)
- **Purpose:** Confirms the problem is real and needs fixing before mechanics spend time

> *An Issue Report says "Engine overheating". Admin creates a Maintenance Ticket. Custodian is assigned to verify. After checking the vehicle, Custodian confirms: "Yes, engine hits 220°F under load." Ticket moves to Verified status. Mechanic can now begin diagnostics.*

Q1.10: What does "Tier-2 Confirmation" mean?

Tier-2 Confirmation is the final approval step where an Admin reviews and approves completed sub-issue repairs:

- **Gate:** A sub-issue cannot move from "For Confirmation" to "Done" without Admin approval
- **Actor:** Admin only (not a Custodian)
- **Purpose:** Ensures repairs meet specifications and quality standards
- **Outcomes:** Approved (sub-issue marked Done) or Rejected (returns to "For Inspection")

> Tier-1 = "Does the problem exist?" Tier-2 = "Is the fix correct?" This two-gate system prevents bad repairs from silently slipping in.

Q1.11: What is the Attestation Checkbox and why is it mandatory?

The Attestation Checkbox is a required confirmation that the person completing a task (Readiness Check, Condition Check, or repair inspection) has actually performed the work and vouches for its accuracy:

- **Legal purpose:** Creates liability trail. The attesting person is accountable for the data
- **Audit purpose:** Prevents rubber-stamping or falsifying records
- **System purpose:** Without it, forms cannot be submitted. It's a guard against accidental incomplete submissions

The checkbox says: "I confirm I have personally performed this work and the information is accurate to the best of my knowledge."

Q1.12: What is a Maintenance Record?

A Maintenance Record is the closed, archived snapshot of completed work on a vehicle. It is created automatically when a Maintenance Ticket is closed:

- **Contains:** Ticket ID, description of work, mechanic who performed it, date range, parts used, cost (if tracked)
- **Purpose:** Historical audit trail. "What maintenance has this vehicle had?"
- **Immutable:** Once created, cannot be edited. Only Admins can view and archive old records

> Maintenance Records are not editable to prevent tampering. They form the vehicle's service history, which is critical for resale value, warranty claims, and compliance audits.

Q1.13: What is Vehicle History and what does it track?

Vehicle History is an immutable audit log of all state changes and significant events for a vehicle:

- **Tracked:** Status changes, Condition changes, Readiness/Condition Check results, tickets opened/closed, location changes, decommissioning, etc.
- **Purpose:** Compliance, debugging, and forensics. "When did this vehicle last pass a Condition Check? Who was driving when the flat tire occurred?"
- **Access:** Read-only for Custodians, full access for Admins

Every action is timestamped and attributed to a user. Deletions don't erase history; they create a "deleted" entry in the log.

Q1.14: What is Deferral and when is it used?

Deferral is a state where a sub-issue's repair work is intentionally postponed:

- **When:** A vehicle is urgently needed, but a repair isn't finished. Deferral allows the vehicle to be dispatched anyway
- **How:** Mechanic or Admin marks a sub-issue "Deferred" instead of "Done"
- **Consequence:** An automatic Issue Report is created as a breadcrumb: "This vehicle has deferred work that needs follow-up"
- **Purpose:** Prevents silent loss of work. The deferred problem doesn't vanish; it becomes a trackable future task

> Deferral is not "give up on the repair." It's "we can't complete it now, but we must remember to finish it later." The automatic Issue Report ensures no one forgets.

Q1.15: What is Fragility Detection (single-point-of-failure)?

Fragility Detection is a system alert that surfaces when a vehicle type has dangerously low availability:

- **Trigger:** When a vehicle type has ≤1 ready unit (e.g., only 1 vehicle out of 5 of that type is Ready status)
- **Dashboard Alert:** A red/orange warning appears: "Fragile: Only 1 ready [vehicle type]. If this unit fails, zero vehicles available"
- **Purpose:** Alerts Admins to single-point-of-failure risk. A catastrophic breakdown would cripple operations
- **Action:** Admin can prioritize repairs on other vehicles of that type to restore redundancy

> *The organization has 3 "Emergency Medical Vehicles". Currently: 1 is Ready, 1 is Under Maintenance, 1 is Out of Service. The dashboard shows a Fragile alert for Emergency Medical Vehicles because only 1 is ready. If that one breaks, zero emergency vehicles are available.*

Q1.16: What is the difference between "Not Fit for Service" condition and "Out of Service" status?

These are two separate concepts:

- **Condition = "Not Fit for Service":** A Condition Check determined the vehicle is mechanically/structurally unsafe to use. Created by a completed Condition Check.
- **Status = "Out of Service":** An Admin decision. The vehicle is temporarily or permanently removed from availability. Status is independent of Condition.

A vehicle with Condition="Not Fit" should have Status="Out of Service", but the statuses are independent. A vehicle could theoretically have Condition="Good" but Status="Out of Service" (e.g., decommissioned due to obsolescence despite being mechanically sound).

Q1.17: What is Vehicle Type and why does it matter?

Vehicle Type is a classification (e.g., "Ambulance", "Fire Truck", "Transport Van") that groups similar vehicles:

- **Purpose:** Organize vehicles by function, enabling role-based dispatch and bulk management
- **Analytics:** Dashboard shows readiness and condition broken down by type
- **Fragility Detection:** Alerts trigger when a type has low availability
- **Preventive Maintenance:** Schedules can be per-type (e.g., "all Ambulances get fluid change every 6 months")

Q1.18: What is a Vehicle Hub/Location and how is it tracked?

A Vehicle Hub is a physical location where vehicles are stationed (e.g., "Central Station", "North Depot", "Field Site A"):

- **Tracking:** Each vehicle has a current Location and a history of past Locations
- **Purpose:** Operational clarity ("Where is Vehicle #47 right now?") and analytics
- **Dashboard:** Shows vehicle distribution across hubs; alerts if vehicles are stranded at unavailable locations

> *A vehicle is stationed at "North Depot". It's dispatched to "Field Site A" for a 3-day mission. The dashboard updates to show it's away. When it returns, the Location is updated back to "North Depot".*

Q1.19: What is Preventive Maintenance and how does it differ from reactive maintenance?

Preventive Maintenance: Scheduled, recurring maintenance to prevent future failures (e.g., "oil change every 6 months").

Reactive Maintenance: Unplanned repairs in response to discovered defects (e.g., "fix the flat tire that was found during Readiness Check").

- **Preventive in VMS:** Created by Maintenance Schedule. Auto-generates a Maintenance Ticket on the scheduled date. When completed, auto-reschedules the next occurrence
- **Reactive in VMS:** Created by Issue Reports and Maintenance Tickets created manually

Preventive maintenance reduces breakdowns; reactive maintenance responds to them. The system supports both.

Q1.20: What is the difference between "Decision Close" and "Done" when closing a ticket?

Both are ways to mark a ticket Closed, but they indicate different outcomes:

- **Done:** All work was completed. All sub-issues are Done. Vehicle is fully repaired
- **Decision Close:** Work is incomplete, but the ticket is intentionally closed anyway. Used when: (a) parts are backordered and vehicle can't wait, (b) a vehicle is declared "Not Fit for Service" and will be scrapped, (c) it's decided the repair is not cost-effective

Both create a Maintenance Record. The record notes which type of close it was. Incomplete work from Decision Close creates automatic Issue Report breadcrumbs (via Deferral) so it's not forgotten.

> *A vehicle needs a new engine (backordered for 6 weeks). The vehicle is urgently needed, so the ticket is Decision Closed with "Needs Engine - Backordered". An automatic Issue Report is created for future follow-up. The vehicle is dispatched with the understanding that it will return for the engine replacement once it arrives.*

Section 2: Feature Explanations (25 Q&A)

Q2.1: How does the Dashboard summarize vehicle readiness and coverage?

The Dashboard presents three primary views:

- **Readiness Panel:** Count of vehicles in Ready status vs. total fleet. Shown by Vehicle Type. "3 of 5 Ambulances Ready"
- **Coverage Panel:** Percentage of fleet with current Condition Checks (not expired). Helps identify vehicles that need condition assessment
- **Fragility Alerts:** Red/orange warnings for vehicle types with ≤1 ready unit

The dashboard aggregates data at the Vehicle Type level, allowing Admins to spot trends: "Emergency Vehicles are at 60% readiness; Transport Vehicles at 95%."

> The dashboard doesn't tell you which specific vehicle to dispatch. That's an operational decision. The dashboard gives you the information to make that decision wisely.

Q2.2: How does the system prevent duplicate Maintenance Tickets for the same issue?

The system does not prevent duplicate tickets automatically. Instead, it relies on human review and workflow discipline:

- **Process:** When an Issue Report is created, Admin reviews existing open tickets for the same vehicle before creating a new ticket
- **Safeguard:** The Issue Report and Ticket screens show related open tickets, making duplicates visible
- **Prevention:** Custodian/Admin can consolidate multiple issues into one ticket with multiple sub-issues if they're related

Example: "Brake system failure" and "Master cylinder leak" are related. Instead of two tickets, create one ticket with two sub-issues.

Q2.3: How are Notifications generated and who receives them?

The system generates notifications on key events:

- **Ticket Assigned:** Mechanic receives notification. "You've been assigned Ticket #42: Engine Repair"
- **Sub-Issue For Inspection:** Assigned Custodian receives notification. "Sub-issue #1 of Ticket #42 is ready for your inspection"
- **Sub-Issue Inspection Result:** Mechanic receives notification. "Your repair on Sub-issue #1 was [Approved/Rejected]"
- **Ticket Closed:** Original reporter receives notification. "Ticket #42 is now closed"
- **Preventive Maintenance Due:** Admin receives notification. "Preventive maintenance for Vehicle #5 is due today"

Notifications ensure no one loses track of their assigned work. The system is push-based (notifications appear in-app); email notifications are a future enhancement.

Q2.4: How does the permission system enforce role-based access?

Every action in the system checks the user's role before allowing it:

- **Frontend Guards:** Buttons, forms, and links are hidden if the user lacks permission (e.g., a Mechanic doesn't see the "Create Issue Report" button)
- **Backend Validation:** All API endpoints verify permissions before returning data or accepting changes
- **Audit Trail:** Permission checks are logged so Admins can see who did what

Examples of permission checks:

- Only Custodians can verify Main Issues
- Only assigned Mechanics can update their ticket progress
- Only Admins can confirm sub-issues and close tickets
- Only Admins can view and export archived records

Q2.5: How does the system ensure data integrity for Maintenance Records?

Maintenance Records are immutable once created:

- **Creation:** Automatically generated when a Maintenance Ticket closes (Done or Decision Close)
- **Freezing:** Records cannot be edited or deleted by anyone, including Admins
- **Archival:** Admins can mark records as archived (hidden from normal views but retained for compliance)
- **Audit Trail:** Each record includes timestamp, actors involved, and exact work performed

> Immutability is non-negotiable for compliance. If records could be edited, a corrupt Admin could hide maintenance failures or fraudulently claim work was completed.

Q2.6: How does the system calculate Vehicle Readiness Percentage?

Readiness % = (Vehicles in Status=Ready) / (Total Vehicles) × 100%

Calculated separately by Vehicle Type:

- Total Ambulances: 5. Ready: 3. Readiness: 60%
- Total Trucks: 8. Ready: 7. Readiness: 87.5%

Vehicles are in Ready status only if:

- Status is "Ready" AND
- Most recent Readiness Check is ≤7 days old AND
- Condition is not "Not Fit for Service"

If any of these conditions fail, the vehicle is excluded from the Ready count.

Q2.7: How are Preventive Maintenance Schedules created and managed?

Preventive Maintenance Schedules are recurring tasks set up by Admins:

- **Creation:** Admin creates a schedule: "Vehicle #3: Oil Change, every 6 months, starting Jan 1"
- **Auto-Generation:** System automatically generates a Maintenance Ticket on Jan 1, July 1, etc.
- **Completion:** When a preventive ticket is closed (Done), the system auto-schedules the next occurrence 6 months later
- **Alerts:** Admin gets a notification when a preventive maintenance is due
- **Overdue:** If the due date passes, the ticket remains open (not auto-skipped) until completed

> *Fluid change scheduled for Vehicle #5 every 6 months. On Jan 1, Ticket #100 is auto-created. On July 1, if Ticket #100 is closed, Ticket #101 is auto-created. If Ticket #100 is still open on July 1, no new ticket is created.*

Q2.8: How does the system track which Condition Check is "current" vs. "past"?

The "current" Condition Check is the most recent one. The system determines this by:

- **Most Recent First:** The API returns Condition Checks in reverse chronological order (newest first)
- **Frontend Marking:** The first record in the list is marked "Current" (green badge). All others are marked "Past" (grey badge)
- **Display:** The dashboard and vehicle detail pages show the Current Condition prominently at the top

There is no separate "current" flag in the database. The recency order determines currency. This prevents ambiguity when a vehicle has multiple condition checks.

> *Vehicle #5 has 3 Condition Checks: (1) Jan 15 = Good, (2) Oct 1 = Needs Maintenance, (3) Dec 1 = Good. The Dec 1 check is Current. The Oct 1 and Jan 15 checks are marked Past. The vehicle's current Condition is "Good" based on the Dec 1 check.*

Q2.9: How does the system handle Vehicle Status transitions?

Vehicle Status has explicit allowed transitions:

- **Ready → Under Maintenance:** When a mechanic is assigned to an active ticket
- **Under Maintenance → Ready:** When all active tickets are closed
- **Ready/Under Maintenance → Out of Service:** Admin decision, always allowed
- **Out of Service → Ready/Under Maintenance:** Admin decision, always allowed
- **Any Status → Decommissioned:** Admin decision, irreversible
- **Decommissioned → Cannot restore:** Decommissioned vehicles stay decommissioned

> The system doesn't force status based on tickets. An Admin can manually move a vehicle to "Out of Service" even if all tickets are closed. This allows for edge cases like "under recall" or "waiting for fuel delivery".

Q2.10: How does the system display and search vehicle and ticket history?

History is displayed in two ways:

- **Vehicle History Page:** Immutable, reverse-chronological log of every status change, check, ticket, and event for a vehicle
- **Ticket History/Archives:** Searchable archive of closed tickets. Filterable by date range, mechanic, vehicle type, status

Search capabilities:

- Find tickets by ticket ID, vehicle ID, or mechanic name
- Filter by date range (e.g., "all tickets closed in Q4")
- Filter by result (e.g., "all tickets Decision Closed")

History is queryable by Admins and Custodians. Maintenance Personnel see only their own tickets.

Q2.11: How are Condition Check results used in the dispatch decision?

A vehicle can only be dispatched if:

- Status = Ready
- Most recent Readiness Check is ≤7 days old and passed
- Most recent Condition Check = "Good" (or at least not "Not Fit for Service")

If any check fails (Condition = "Needs Maintenance"), the vehicle CAN still be dispatched if Status is Ready and the operational need overrides the warning. The dashboard alerts "Condition: Needs Maintenance" but doesn't block dispatch.

> Dispatch decisions are ultimately human decisions. The system provides information ("this vehicle needs maintenance") but doesn't force outcomes. An emergency dispatch might proceed with a "Needs Maintenance" vehicle if it's critical.

Q2.12: How does the SmartForm component handle CRUD operations across the app?

SmartForm is a centralized React component that handles Create, Read, Update, Delete for all entities:

- **Pattern:** Admin opens "Create Vehicle" → SmartForm renders with required fields (name, type, location, etc.) → User submits → API call → Success/error message
- **Configuration:** Each entity (Vehicle, Ticket, Condition Check, etc.) has a SmartForm config specifying fields, validation rules, and API endpoints
- **Error Handling:** Validation errors are displayed inline on fields. API errors show as toast messages with auto-dismiss
- **State Management:** SmartForm manages form state (dirty fields, touched fields, errors) internally using React hooks

By centralizing CRUD logic, the app avoids code duplication and ensures consistent UX across all forms.

Q2.13: How does the loading overlay system provide feedback during form submissions?

When a form is submitted, a full-screen loading overlay appears:

- **Display:** Semi-transparent dark overlay covering the entire screen
- **Content:** Centered card with rotating gear icon and "Loading..." text
- **Duration:** Remains until the API responds (success or error)
- **Non-blocking:** Users can't interact with the page while loading; all buttons/forms are disabled under the overlay

This prevents double-submission and makes clear that work is in progress.

> *User submits "Create Maintenance Ticket". Loading overlay appears. After 2 seconds, API responds with success. Overlay disappears. Success message appears: "Ticket #42 created".*

Q2.14: How are error messages and validation feedback displayed?

Errors appear in two contexts:

- **Form Validation:** Inline, red error text below each field (e.g., "Vehicle type is required")
- **API Errors:** Toast notification at the bottom of the screen, auto-dismisses after 5-8 seconds. Examples: "Ticket #42 not found", "You don't have permission to close this ticket"
- **Multi-line Errors:** Messages with more than one line auto-dismiss after 8 seconds. Single-line messages auto-dismiss after 5 seconds

Error messages are specific and actionable: "Readiness Check cannot be marked as "Not Ready" without a reason. Please fill in the Reason field."

Q2.15: How does the backend API enforce business logic constraints?

The backend (Laravel 13) validates all requests before processing:

- **State Machine:** API prevents invalid state transitions (e.g., "Cannot move sub-issue from Done to Under Repair")
- **Permission:** API checks user role before allowing actions (e.g., "Only Custodians can verify Main Issues")
- **Data Integrity:** API validates foreign keys (e.g., "Vehicle ID 999 does not exist")
- **Business Rules:** API enforces rules like "Readiness Check must be marked as "Not Ready" with a reason"

If validation fails, the API returns a 422 error with detailed field-level errors. Frontend displays these to the user.

Q2.16: How does the system support light and dark mode?

The app uses CSS custom properties (CSS variables) to define colors for light and dark themes:

- **Auto-Detection:** On first load, the system checks the OS preference (light or dark)
- **Manual Override:** User can toggle between light and dark mode via a button in the header
- **Persistence:** User preference is saved in localStorage and restored on next login
- **CSS Implementation:** Colors are defined via custom properties on the :root element, overridden in @media (prefers-color-scheme: dark) and further overridden by [data-theme="light"]/[data-theme="dark"] attributes

All UI components respect the current theme. Charts, tables, and modals automatically adjust colors.

Q2.17: How are Ticket Archives accessed and managed?

Closed tickets are automatically archived:

- **Auto-Archival:** When a ticket is closed (Done or Decision Close), it's moved to the archives. No manual action needed
- **Access:** Admins and Custodians can view archived tickets via "Ticket Archives" section
- **Searchable:** Archives are filterable and searchable by ticket ID, vehicle, date range, mechanic, etc.
- **Export:** Admins can export archived data as CSV for reporting or external audits

Archived tickets are read-only. No one can edit a closed ticket.

Q2.18: How does the system handle Role-Based Visibility of data?

Each role sees different data:

- **Admin:** Full visibility. All vehicles, tickets, checks, users, reports
- **Custodian:** Own assigned vehicles/tickets. Can see all vehicles but only edit/inspect their own tickets
- **Maintenance Personnel:** Only their assigned tickets. Cannot view other mechanics' work

This is enforced at the API level. If a Mechanic requests "GET /tickets/999" where ticket 999 is not assigned to them, the API returns 403 Forbidden.

Q2.19: How does the system log and audit all changes?

Every significant action is logged:

- **What:** Action taken (created vehicle, closed ticket, verified issue, etc.)
- **Who:** User ID and name of the person performing the action
- **When:** Timestamp to the minute
- **Data:** Before/after values (e.g., "Status changed from Ready to Under Maintenance")

Logs are immutable and viewable by Admins only. Used for compliance audits, debugging, and investigating discrepancies.

Q2.20: How does the vehicle intake process work for new or returned vehicles?

When a vehicle enters the fleet (new, returned from long-term storage, or repaired after decommission):

- **Step 1:** Admin creates the vehicle record (name, type, location, year, VIN, etc.). Status defaults to "Out of Service"
- **Step 2:** Custodian performs an initial Condition Check. Result determines initial Condition status
- **Step 3:** If Condition = "Good", Custodian performs a Readiness Check. If passes, Status can change to "Ready"
- **Step 4:** If any issues are found, Issue Reports are created and Maintenance Tickets follow the standard repair workflow

The vehicle remains "Out of Service" until both Condition and Readiness checks pass.

Q2.21: How does the system calculate and track mileage and usage metrics?

Mileage tracking is **out of scope** for the current VMS. The system does not track:

- Vehicle odometer readings
- Distance traveled per mission
- Fuel consumption
- Mileage-based maintenance triggers (e.g., "oil change every 5,000 miles")

Preventive maintenance is triggered by time (calendar-based) only, not mileage. Future versions may add mileage tracking if operational needs require it.

Q2.22: How are Maintenance Personnel assigned to tickets and what are their responsibilities?

Assignment:

- Admin or Supervisor manually assigns a Maintenance Ticket to a mechanic
- Mechanic receives a notification: "You've been assigned Ticket #42"
- Mechanic updates the ticket status to "Active" when work begins

Responsibilities:

- Diagnose the problem
- Create sub-issues for each task (replace part, test system, etc.)
- Update sub-issue status as work progresses: Open → Under Repair → For Inspection
- Cannot close sub-issues themselves (requires Custodian inspection and Admin confirmation)

Q2.23: What reports are available to Admins and what insights do they provide?

Available Reports:

- **Readiness Report:** Vehicle readiness % by type over time. Identifies trends (e.g., "Ambulances dropping from 80% to 60%")
- **Maintenance Report:** Tickets closed, average time-to-close, common issues by vehicle type
- **Condition Report:** Vehicles in each condition state; vehicles overdue for condition checks
- **Mechanic Report:** Work completed per mechanic, average repair time, quality metrics (% of tickets rejected on first inspection)

Reports are exportable as CSV for use in spreadsheets or external analytics tools.

Q2.24: How does the system handle vehicle decommissioning and restoration?

Decommissioning:

- Admin marks a vehicle Status = "Decommissioned" (e.g., vehicle is obsolete, salvaged, or retired)
- Decommissioned vehicles are hidden from the Ready count
- History is preserved for audit purposes

Restoration:

- A decommissioned vehicle cannot be restored to Ready directly
- Admin must create a new vehicle record if the old one is needed again (different history)
- OR: Admin manually changes Status from "Decommissioned" to "Out of Service", then requires a fresh intake (Condition and Readiness checks)

> *A vehicle was decommissioned in 2024. In 2025, the organization buys it back. Admin restores it to "Out of Service". A new Condition Check is required before it can be marked Ready.*

Q2.25: How does the Custodian inspection process work for sub-issue repairs?

Workflow:

- **Step 1:** Mechanic marks sub-issue "For Inspection"
- **Step 2:** Assigned Custodian receives notification and reviews the repair
- **Step 3:** Custodian verifies: "Does the repair meet specifications? Is it safe?" and marks "Approved" or "Rejected"
- **Step 4:** If Approved, sub-issue moves to "For Confirmation" (awaiting Admin final sign-off)
- **Step 5:** If Rejected, sub-issue returns to "For Inspection" for mechanic to rework

Custodians act as quality gates. Their sign-off prevents bad repairs from being marked Done.

Section 3: Tricky & Edge Case Questions (20 Q&A)

Q3.1: A vehicle passes its Condition Check ("Good") but fails its Readiness Check ("Not Ready"). Can it be dispatched?

No. The vehicle cannot be dispatched. Readiness Check failure is a blocker.

Why? Condition checks structural/mechanical integrity. Readiness checks consumables. A vehicle with "Good" condition but low fuel, flat tire, or missing equipment is unsafe to dispatch.

Resolution: The issue identified in the Readiness Check must be fixed. Examples:

- Low fuel → Refuel before dispatch
- Flat tire → Repair/replace tire before dispatch
- Missing equipment → Install equipment before dispatch

Once the issue is fixed, a new Readiness Check is performed. If it passes, the vehicle can be dispatched.

> Condition checks the "frame"; Readiness checks the "fuel tank". Both must pass. A beautiful car with an empty fuel tank still can't go anywhere.

Q3.2: Can a Maintenance Ticket be closed if there is unfinished work on sub-issues?

Yes, via "Decision Close". But unfinished work creates automatic Issue Report breadcrumbs to ensure it's not forgotten.

Scenario: A vehicle needs a new transmission (backordered for 8 weeks). The vehicle is urgently needed. Mechanic defers the transmission replacement work and closes the ticket as "Decision Close".

System Response:

- Ticket moves to Closed status
- Automatic Issue Report is created: "Vehicle #5 has deferred work: Replace transmission"
- When the part arrives, Admin creates a new ticket based on the Issue Report

Decision Close is valid when:

- Parts are backordered and vehicle can't wait
- Vehicle is being decommissioned (no more repair needed)
- Repair is deemed not cost-effective
- Operational urgency overrides completion

> Decision Close with deferred work is better than keeping a ticket open indefinitely. It signals "this work is paused, not abandoned" and creates an automatic reminder.

Q3.3: What is the difference between a Custodian rejecting a ticket and an Admin reopening a closed ticket?

Custodian Rejection (of Main Issue):

- Occurs before mechanic work starts
- Custodian verifies the Main Issue and determines it doesn't exist or is cosmetic
- Ticket closes immediately, no work is done
- Example: "Reported 'engine overheating', but I checked the temperature gauge. It's normal. This is a false alarm."

Admin Reopening (of closed ticket):

- Occurs after a ticket is closed (Done or Decision Close)
- Admin discovers the repair was inadequate or new issues arose
- Ticket can be reopened to reopen sub-issues or request additional work
- Example: "We closed the transmission rebuild 3 weeks ago, but now it's leaking again. Reopening the ticket for the mechanic to revisit."

Why different? Rejection prevents unnecessary work upfront. Reopening fixes work that was supposed to be complete but failed.

Q3.4: How does the system prevent duplicate Maintenance Tickets for the same issue?

The system does NOT prevent duplicates automatically. Prevention relies on human process:

- **Before Creating:** Admin checks existing open tickets for the same vehicle
- **Visual Hint:** The ticket creation form shows a list of open tickets for the vehicle. Admin can consolidate related issues into one ticket with multiple sub-issues
- **Naming Convention:** If tickets must be separate, they're named descriptively to avoid confusion (e.g., "Ticket #100: Brake System" and "Ticket #101: Cooling System")

Scenario: Vehicle #5 has "flat tire" and "rust on doors". Instead of two tickets, create one with two sub-issues: (1) Replace tire, (2) Sand and repaint door.

> The system encourages consolidation by making open tickets visible. A future enhancement could auto-prevent duplicates, but for now it's a workflow discipline issue.

Q3.5: What happens if a Maintenance Personnel is reassigned mid-repair or becomes unavailable?

Scenario: Mechanic A is assigned to Ticket #42 (transmission rebuild). Midway through (sub-issue still "Under Repair"), Mechanic A gets sick.

System Response:

- Admin reassigns the ticket to Mechanic B
- Mechanic B receives notification: "You've been assigned Ticket #42 (in progress)"
- Mechanic B can see the work history and prior notes
- Mechanic B continues the sub-issue from where it was (Under Repair)

If reassignment is permanent:

- All of Mechanic A's open tickets are reassigned to other available mechanics
- Admin ensures no work is lost during the handoff

Data Integrity: The audit trail shows both mechanics worked on the ticket. History is preserved for accountability.

Q3.6: Why does Readiness Check expire after 7 days, but Condition Check doesn't expire?

Readiness Expiration Logic: Consumables (fuel, fluids, air in tires, batteries) deplete or degrade over time. A vehicle ready on Monday may run low on fuel by Friday. A 7-day expiration ensures freshness checks before dispatch.

Condition Non-Expiration Logic: Mechanical/structural defects don't fix themselves over time. If the transmission was "Good" in October, it's still "Good" in December unless something broke it. There's no reason to re-check it.

Exception: If a defect is discovered AFTER the last Condition Check, an Issue Report is filed and a new check may be ordered. But the prior check result doesn't "expire."

> *Vehicle checked in January: Condition="Good", Readiness="Ready". In February: Readiness Check required (7 days old). Condition Check is still valid (Good). In March: Vehicle hits a pothole. Issue Report filed. New Condition Check ordered to assess damage.*

Q3.7: Can an Issue Report exist without a corresponding Maintenance Ticket?

Yes. An Issue Report is the complaint; a Maintenance Ticket is the action plan. Not every complaint becomes a ticket.

Scenario: Custodian finds a slow oil leak (≤1 drop per hour). Creates an Issue Report. After review, Admin decides: "This is minor. Let's monitor it without fixing it yet." No ticket is created.

When no ticket is created:

- Issue Report stays open as a "watch item"
- On the next Readiness/Condition Check, if the leak worsens, a ticket is created
- Or Issue Report is closed as "monitored"

When a ticket IS created:

- Issue Report is linked to the ticket
- Both are visible in the workflow

> Issue Reports are low-cost to create (just a description). Tickets are high-cost (assign mechanic, do work). The system captures all problems, but only converts significant ones to tickets.

Q3.8: What happens when a required part is backordered and the vehicle is urgently needed?

Scenario: Vehicle #7 needs a new engine. Part is backordered for 6 weeks. But the vehicle is urgently needed for a 2-week mission.

Solution: Decision Close with Deferral

- Mechanic updates the sub-issue: "Engine replacement - Backordered, Part XYZ ordered, ETA 6 weeks"
- Mechanic marks sub-issue "Deferred" instead of "Done"
- Mechanic closes ticket as "Decision Close" with note: "Waiting for part arrival"
- System auto-creates Issue Report: "Vehicle #7 pending engine replacement"
- Vehicle Status changes to "Ready" (work paused, vehicle available for dispatch)

When part arrives:

- Admin reviews the Issue Report
- Creates a new ticket for the engine replacement
- Mechanic completes the work

> Deferral allows operational flexibility (dispatch the vehicle) without losing track of pending work (auto-created breadcrumb ensures follow-up).

Q3.9: How does deferral prevent "silent work loss" and ensure follow-up?

The Problem: Without deferral, a mechanic might close a ticket with incomplete work and forget to come back. The vehicle runs, no one notices, and the deferred problem is never addressed.

The Solution: Automatic Issue Report Breadcrumb

- When a sub-issue is marked "Deferred", the system automatically creates a new Issue Report with the deferred work description
- Issue Report appears in the Issues queue with a flag: "Requires Follow-up"
- Admin reviews the flag and creates a new ticket when the vehicle returns or conditions permit

Audit Trail: The deferred work is traceable. Admin can search for all deferred sub-issues and see which vehicles have pending work.

> *Mechanic defers an "alignment check" because the alignment machine is broken. Sub-issue marked Deferred. Issue Report auto-created: "Vehicle #3 alignment pending - equipment malfunction". Two weeks later, equipment is fixed. Admin searches "Deferred issues" and finds it. Creates new ticket. Work is completed.*

Q3.10: Can a non-assigned Custodian inspect a ticket that was assigned to a different Custodian?

No. A sub-issue can only be inspected by the Custodian it was assigned to.

Why? Accountability. If Custodian A verified the Main Issue, they should be the one inspecting repairs. Requiring the same Custodian creates continuity.

Exception: Admin can inspect any ticket (Admins have all permissions).

Scenario: Ticket assigned to Custodian A. Sub-issue ready for inspection. Custodian B tries to inspect. System blocks with: "This ticket is assigned to Custodian A. Only Custodian A can inspect it (or an Admin)."

If Custodian A is unavailable: Admin can reassign the ticket to Custodian B, or Admin can inspect directly.

Q3.11: What happens if a vehicle is in "Under Maintenance" status but has no active open tickets?

Scenario: Vehicle #4 is Status="Under Maintenance". All tickets were just closed. But Status hasn't been manually updated to "Ready".

System Response:

- The system does NOT auto-transition status. An Admin must manually change it
- Vehicle remains "Under Maintenance" until explicitly changed
- This prevents accidental dispatch of vehicles that were recently under maintenance

Admin Process:

- Verify all tickets are closed
- Verify vehicle passed final Readiness Check
- Manually change Status to "Ready"

> The system is conservative. It requires human confirmation before marking a vehicle ready after maintenance, preventing accidental dispatch of partially repaired vehicles.

Q3.12: How does the system detect and alert on "fragility" (single-point-of-failure)?

Fragility Condition: A vehicle type has ≤1 ready unit (e.g., only 1 out of 5 Emergency Vehicles is Ready).

Alert Mechanism:

- Dashboard shows a red/orange "FRAGILE" badge on vehicle type tiles
- Alert text: "Fragile: Only 1 ready [Vehicle Type]. If this unit fails, zero vehicles available"
- Admin notification: "Emergency Vehicles are fragile—prioritize repairs on the other 4 units"

Why it matters: If the only ready vehicle breaks down, the organization has zero operational vehicles of that type. This is a critical risk.

Admin Response: Prioritize repairs on the Under Maintenance/Out of Service vehicles to restore redundancy.

> *Fire Department has 3 Ladder Trucks. One is Ready, two are Under Maintenance. Dashboard shows "FRAGILE" alert. Admin prioritizes finishing the maintenance to get a second truck ready. Once 2 are ready, the fragile alert disappears.*

Q3.13: Can an Issue Report be created for a vehicle that is in "Out of Service" or "Decommissioned" status?

Yes, technically. But it's unusual:

- **Out of Service:** Vehicle is temporarily unavailable (e.g., waiting for fuel). Issue Reports can be created; when Status changes to Ready, tickets can be created
- **Decommissioned:** Vehicle is permanently retired. Creating Issue Reports is wasteful, but system doesn't prevent it. Usually Admins won't bother creating tickets for decommissioned vehicles

Better Practice: Don't create Issue Reports for decommissioned vehicles. Close the vehicle record and move on.

Q3.14: What happens if a Custodian inspects a sub-issue and rejects it? Does the mechanic have to start over?

No, not necessarily. The sub-issue returns to "For Inspection" state with rejection feedback. Mechanic can rework it without starting from zero.

Workflow:

- Sub-issue marked "For Inspection"
- Custodian inspects, finds issue (e.g., "Weld is not clean") and marks "Rejected"
- Sub-issue returns to "For Inspection" state
- Mechanic receives notification: "Your repair on Sub-issue #1 was Rejected. Reason: Weld is not clean"
- Mechanic reworks the weld, re-marks "For Inspection"
- Custodian re-inspects. If approved, sub-issue moves to "For Confirmation"

> Rejections are rework cycles, not restarts. The sub-issue stays "For Inspection" until Custodian approves. This can happen multiple times until quality is met.

Q3.15: If an Admin overrides a Custodian's "Not Ready" Readiness Check verdict, what happens?

Scenario: Custodian marks Readiness="Not Ready" (low fuel). Vehicle is urgently needed for emergency. Admin says "I'll allow it anyway."

System Response:

- The system does NOT allow override of Readiness Check verdicts
- A "Not Ready" verdict is binding. The vehicle cannot be dispatched
- If an Admin disagrees, they must:
  - Fix the issue (refuel the vehicle)
  - Re-run the Readiness Check
  - Get a "Ready" verdict
  - THEN dispatch the vehicle

Why? Readiness verdicts are safety gates. Allowing overrides creates liability and risk.

Q3.16: Can a ticket be in "Active" state with no assigned mechanic?

The system logic enforces: A ticket must have an assigned mechanic before moving to "Active" state.

Workflow:

- Ticket created in "Open" state (no mechanic yet)
- Admin assigns a mechanic (e.g., Mechanic A)
- Mechanic marks ticket "Active" when work begins

A ticket cannot jump to "Active" without an assignee. This ensures accountability—someone owns the work.

Q3.17: What prevents a Maintenance Ticket from staying "Open" forever?

The system does NOT prevent open tickets from aging. Tickets stay "Open" until someone acts on them.

Safeguards:

- **Visibility:** The dashboard shows a count of open tickets. Admins can see "30 open tickets, oldest is 6 months old"
- **Sorting:** Ticket list can be sorted by age, highlighting old tickets for review
- **Admin Responsibility:** Admins must periodically review old tickets and either assign them, cancel them, or close them

Why not auto-close? Auto-closing would hide important work. Better to surface it visibly and let Admins decide.

Q3.18: Can the system prevent vehicles of a critical type from all entering "Under Maintenance" simultaneously?

No. The system does NOT prevent all units of a type from entering maintenance at the same time.

What it does: Alerts via Fragility Detection when only ≤1 unit is Ready. This is a warning, not a prevention.

Admin responsibility: Stagger maintenance or defer some repairs to maintain operational redundancy.

Scenario: 3 Ambulances. Ideally, 2 should be Ready, 1 Under Maintenance. If all 3 go Under Maintenance, the dashboard shows "FRAGILE - zero ready". Admin recognizes this is unsustainable and prioritizes finishing at least one ambulance to restore service.

Q3.19: How does the system handle a situation where the assigned Custodian of a ticket is no longer employed?

Scenario: Custodian A was assigned to verify/inspect Ticket #42. Custodian A leaves the organization.

System Response:

- The system does NOT auto-reassign tickets when a user is deleted
- Admin must manually reassign the ticket to a new Custodian (Custodian B)
- Custodian B receives notification and can now inspect the work

Data Preservation: The audit trail still shows Custodian A was originally assigned. This is important for accountability and compliance.

Q3.20: Can a vehicle be marked "Not Fit for Service" if all recent tickets are closed?

Yes. The Condition Check verdict is independent of ticket status.

Scenario: Vehicle #8 passed repairs (ticket closed). But during a routine Condition Check, the inspector discovers severe rust in the frame. Inspector marks Condition="Not Fit for Service".

Consequence:

- Vehicle Status may need to change to "Out of Service"
- New Issue Report is created: "Rust discovered, vehicle not fit for service"
- New Maintenance Ticket may be created to address the rust
- Or, if repair isn't cost-effective, vehicle is decommissioned

Condition Checks can happen anytime, not just after tickets close. They're continuous quality gates.

Section 4: Design Decisions & Architecture (12 Q&A)

Q4.1: Why was Laravel 13 + React 19 + Vite chosen for the VMS architecture?

Backend: Laravel 13

- Mature PHP framework with built-in: authentication, authorization, ORM (Eloquent), migrations, validation
- Strong at: REST APIs, database transactions, business logic enforcement
- Great ecosystem: Packages for notifications, file handling, logging
- Decision: Separates API (backend) from UI (frontend), allowing independent scaling and team assignments

Frontend: React 19 + Vite

- React: Component-based UI, state management, large developer community
- Vite: Fast build tool (vs. Create React App), excellent DX for development
- SPA (Single Page App): Page loads once, subsequent navigation is fast (no full-page reloads)

Communication: RESTful JSON API

- Backend exposes endpoints (GET /vehicles, POST /tickets, etc.)
- Frontend consumes them and renders UI
- Clear separation of concerns

Q4.2: Why is the state machine architecture (Ticket Open → Active → Closed, Sub-Issue Open → Under Repair → For Inspection → For Confirmation → Done) better than a flat approach?

State Machine Benefits:

- **Prevents Invalid Transitions:** A sub-issue can't jump from Open to Done without inspection (For Inspection enforces review)
- **Enforces Process:** The sequence (Under Repair → For Inspection → For Confirmation) ensures work is reviewed at each step
- **Clarity:** Everyone knows what "For Inspection" means. It's not ambiguous
- **Audit Trail:** Each transition is timestamped, creating a clear history

Flat Approach (No State Machine):

- Would allow: Create sub-issue → Mark Done (skipping inspection)
- Quality escapes: Unchecked repairs could be silently marked complete
- Process violations: No enforcement of review gates

Conclusion: The state machine is a guardrail that prevents shortcuts and ensures quality.

Q4.3: Why allow overlapping RBAC roles instead of a strict hierarchy (e.g., Admin > Custodian > Mechanic)?

Overlapping Roles (Current Design):

- Admin can do Admin + Custodian + Mechanic tasks
- Custodian can do Custodian + some Admin tasks
- Mechanic can do Mechanic tasks only

Why overlapping?

- **Flexibility:** In small teams, one person might wear multiple hats. Overlapping roles accommodate this
- **Separation of Duties:** Even with overlap, key decisions have gates (e.g., Admin must confirm tickets, not Custodian). This prevents a single role from controlling the entire workflow
- **Operational Reality:** Organizations don't fit neat hierarchies. This design is pragmatic

Strict Hierarchy (Not Chosen):

- Would require Admins to do Custodian work, which is inefficient
- Would break in scenarios where Admins need to fill in (sick leave, vacations)

Q4.4: Why is there a separate Readiness Check and Condition Check instead of combining them?

Separate Checks (Current Design):

- **Readiness:** Quick (5 min), consumables-focused, time-expiring (7 days). Answers: "Can we dispatch this right now?"
- **Condition:** Thorough (30 min - 2 hours), mechanical/structural, permanent until changed. Answers: "Is this vehicle safe to operate at all?"

Why separate? Different frequency, scope, and teams. Combining would create:

- Long checks before every dispatch (inefficient)
- Unnecessary re-checking of mechanical items that don't change
- Confusing semantics ("Does this mean structural or consumables?")

Analogy: An airplane has pre-flight checks (fuel, instruments) and periodic overhauls (engine rebuild). You don't do an overhaul before every flight.

Q4.5: Why are Maintenance Records immutable?

Immutability Rationale:

- **Compliance:** Records are evidence of work performed. Editability creates liability and audit failures
- **Fraud Prevention:** A corrupt Admin couldn't falsify records (e.g., "Claim we did maintenance when we didn't")
- **Historical Accuracy:** Resale value, warranty claims, and compliance audits depend on accurate history

Exception: Archival Records can be marked archived (hidden from normal views) but never deleted or edited.

Correction Workflow: If a record is created with wrong data, a new corrective record is created, not an edit to the original.

Q4.6: Why is Tier-1 (Custodian) verification required before Tier-2 (Admin) confirmation?

Two-Gate Design:

- **Tier-1 (Custodian):** "Does the problem described in the ticket actually exist?" Custodian with vehicle knowledge verifies the Main Issue
- **Tier-2 (Admin):** "Is the fix correct?" Admin with authority approves completed repairs

Why both?

- Prevents false alarms: If the Main Issue doesn't exist, work is skipped before it starts
- Prevents bad repairs: If Tier-1 is skipped, a mechanic might do unnecessary or incorrect work
- Prevents low-quality repairs: If Tier-2 is skipped, bad repairs could be marked complete

Result: A three-perspective check (Custodian says problem exists → Mechanic fixes it → Admin says it's fixed well) is more robust than one or two perspectives alone.

Q4.7: Why does the system create automatic Issue Report breadcrumbs when work is deferred?

Problem Solved: Without breadcrumbs, deferred work could be forgotten. A mechanic might close a ticket with pending repairs and never come back.

Solution: Automatic Issue Report

- When a sub-issue is marked "Deferred", system auto-creates an Issue Report with the deferred work description
- Issue Report appears in the active queue, making it visible to Admins
- Admin can track deferred work and create new tickets when conditions permit

Why automatic? Manual breadcrumbs would require the mechanic to remember to create one, which defeats the purpose. Automatic ensures nothing is lost.

Q4.8: Why are Vehicle Status and Condition independent rather than computed from each other?

Independence Rationale:

- **Status:** Operational policy decision. "Can this vehicle be dispatched right now?" Controlled by human decisions
- **Condition:** Physical state. "What does the last inspection tell us about this vehicle?" Determined by inspection results

Why not compute Status from Condition? Because they answer different questions:

- A vehicle might be Condition=Good but Status=Under Maintenance (mechanic is doing preventive work)
- A vehicle might be Condition=Needs Maintenance but Status=Ready (management decided to dispatch it anyway, accepting the risk)
- A vehicle might be Condition=Good but Status=Out of Service (waiting for fuel delivery, broken GPS, etc.)

Result: Independence allows operational flexibility while maintaining independent data quality.

Q4.9: Why does the VMS not track mileage or fuel consumption?

Out of Scope Decision: Mileage tracking was explicitly excluded from the VMS scope during project planning.

Why?

- Mileage-based maintenance (e.g., "oil change every 5,000 miles") requires odometer readings at every dispatch, return, and check
- This adds operational burden to every dispatch/return workflow
- The current organization relies on time-based maintenance (e.g., "oil change every 6 months"), not mileage

Future Enhancement: If operations require mileage tracking, it would be added as a separate module with its own workflows.

Current Workaround: Admins can record mileage in ticket notes or external systems if needed.

Q4.10: Why is inspection a separate step from mechanic work instead of having mechanics self-verify?

Separation of Duties: A mechanic who both does the work AND verifies it creates a conflict of interest ("I'm done, I must approve this").

Three-Party Model (Better):

- Mechanic performs work (Under Repair)
- Custodian inspects work (For Inspection → Approved/Rejected)
- Admin confirms work (For Confirmation → Done)

Quality Result: No single party controls the entire flow. Each has a specific responsibility.

Alternative (Not Used): Self-Verification Would allow mechanic to mark repairs complete without inspection, risking low-quality work slipping through.

Q4.11: How does the Fragility Detection algorithm work and why is it important?

Algorithm:

- For each Vehicle Type: Count = (Vehicles with Status=Ready)
- If Count ≤ 1: Trigger FRAGILE alert
- Dashboard shows: "Fragile: Only [Count] ready [Vehicle Type]"

Why Important:

- Identifies critical risk. If a type has only 1 ready unit and it breaks, zero units are available
- Alerts Admins to stagger maintenance or prioritize repairs on other units
- Prevents silent operational collapse (e.g., "All 3 ambulances down simultaneously")

Threshold: ≤1 ready unit is the trigger. This is conservative (flags when < 2 ready), which is appropriate for critical equipment.

Q4.12: Why use a portal-based overlay system for loading states instead of simple spinners?

Portal Pattern (Current): Creates a full-screen overlay that appears above all other content, preventing user interaction

- **Benefit:** Users can't accidentally submit the form twice or click other buttons while waiting
- **Clear Feedback:** Full screen coverage makes it obvious that something is loading
- **Prevents Double-Submission:** All buttons are inaccessible under the overlay

Simple Spinner (Not Used): Would show loading in a corner while form remains interactive

- Risk: User clicks submit again, creating duplicate tickets
- Confusion: Not obvious that the system is processing

Conclusion: Portal overlays are more robust for critical operations (form submissions, ticket workflows).

Section 5: Scenario-Based Questions (10 Detailed Scenarios)

Scenario 1: Flat Tire Discovered During Readiness Check (Before Dispatch)

Setup: Vehicle #3 is scheduled for dispatch today (2-hour mission). Custodian A performs a routine Readiness Check.

Discovery: Left front tire is flat.

System Actions:

1. Custodian marks Readiness Check result: "Not Ready" with reason "Flat tire - left front"
2. System prevents dispatch (Readiness = Not Ready blocks dispatch)
3. Custodian creates Issue Report: "Flat tire - Vehicle #3"

Next Steps:

1. Admin reviews Issue Report and decides: (a) Quick fix (replace tire in 30 min), or (b) Defer mission
2. If Quick Fix: Mechanic replaces tire. Custodian performs new Readiness Check. If passes, dispatch proceeds
3. If Defer: Mission is rescheduled. Ticket is created for tire replacement later

Key Insight: Readiness Check caught the issue before dispatch, preventing a mission failure in the field. The system worked as designed.

Scenario 2: Mechanic Discovers Secondary Problem During Repair

Setup: Ticket #100 is assigned to Mechanic A to replace a water pump. Work is "Under Repair".

Discovery: While removing the pump, Mechanic A discovers the radiator is corroded and needs replacement too.

System Actions:

1. Mechanic A creates Sub-Issue #2: "Replace radiator - corrosion" (in addition to existing Sub-Issue #1: "Replace water pump")
2. Both sub-issues are now "Under Repair" on the same ticket
3. Mechanic A continues with both repairs

Prevention of Scope Creep: The system enforces that all sub-issues are documented. This prevents:

- Undocumented work (mechanic silently fixes issues without records)
- Cost overruns (Admin can see the expanded scope and approve/reject the additional work)
- Incomplete documentation (the repair history shows exactly what was done)

Admin Review: Admin sees Ticket #100 now has 2 sub-issues. Admin can approve both or request Mechanic to prioritize and defer the radiator work.

Scenario 3: Mechanic Becomes Unavailable Mid-Repair

Setup: Mechanic A is assigned to Ticket #42 (transmission rebuild). Work is "Under Repair" on Sub-Issue #1.

Event: Mechanic A calls in sick, will be out for 2 weeks.

System Actions:

1. Admin reassigns Ticket #42 to Mechanic B
2. Mechanic B receives notification: "You've been assigned Ticket #42 (in progress)"
3. Mechanic B can view the work history, prior notes, and current status (Under Repair)

Handoff: Mechanic B reviews:

- What has been done so far (Sub-Issue #1: "Replace gaskets" is 80% done)
- What remains (Sub-Issue #2: "Replace seals", Sub-Issue #3: "Pressure test")
- Any notes or warnings from Mechanic A

Continuation: Mechanic B continues Sub-Issue #1 from where Mechanic A left it. When complete, Mechanic B moves to Sub-Issues #2 and #3.

Audit Trail: The history shows both Mechanic A and Mechanic B worked on the ticket. Timeline shows Mechanic A until Day 5, then Mechanic B from Day 6 onward.

Scenario 4: Custodian Inspection Finds "No Issues" but Admin is Skeptical

Setup: Ticket #67 involved a "noisy transmission" repair. Mechanic completed work. Sub-issue is "For Inspection".

Custodian Action: Custodian A test-drives the vehicle, finds no noise, marks inspection "Approved".

Sub-Issue Result: Sub-issue moves to "For Confirmation" (waiting for Admin final sign-off).

Admin Review: Admin questions: "Did the transmission really stop making noise, or did the mechanic just mask the symptom?" Admin marks confirmation "Rejected" with note: "Requires extended test drive—15 minutes minimum at highway speeds".

System Response: Sub-issue returns to "For Inspection". Custodian A receives rejection notification and performs a more thorough test drive (highway speeds, load, etc.). Finds noise has returned at 65 mph.

Resolution: Issue Report is created: "Transmission repair incomplete—noise returns at highway speeds". Mechanic investigates further (incorrectly diagnosed root cause). New Sub-Issue is created for deeper diagnostics.

Key Insight: The two-gate system (Custodian inspection + Admin confirmation) prevents low-quality repairs. Admin's skepticism triggered the discovery of inadequate work.

Scenario 5: Decision Close with Incomplete Work and "Not Fit" Condition

Setup: Vehicle #12 needs major engine diagnostics (estimated 3 weeks). However, the vehicle is flagged as "high value" and urgently needed for a critical 1-week operation.

Admin Decision: "We must dispatch this vehicle despite incomplete repairs. I'll close the ticket as Decision Close and create a breadcrumb for future follow-up."

Actions:

1. Admin marks Sub-Issue #1 "Deferred" with note: "Engine diagnostics incomplete—scheduled for completion upon vehicle return"
2. Admin closes Ticket #100 as "Decision Close" with outcome: "Not Fit for Service—Deferred pending diagnostics completion"
3. System auto-creates Issue Report: "Vehicle #12 pending engine diagnostics—high priority"
4. Vehicle Status changes to "Ready" (for dispatch), but Condition stays "Not Fit for Service"

Risk Communication: Dashboard shows:

- Vehicle #12 is Ready (available for dispatch)
- Vehicle #12 is Condition = Not Fit (proceed with caution)
- Open Issue Report: "Pending engine diagnostics"

Future: Vehicle returns after 1 week. Admin creates new Ticket #101 based on the Issue Report. Mechanic completes engine diagnostics. If significant issues found, vehicle may be removed from service.

Key Insight: Decision Close with Deferral allows operational flexibility (dispatch the vehicle) without losing track of unfinished work (auto-created breadcrumb ensures follow-up).

Scenario 6: Multiple Vehicles of Same Type Under Maintenance Simultaneously

Setup: Organization has 4 "Transport Vans". Operations need 2 ready at all times (redundancy). Currently:

- Van #1: Status=Ready, Condition=Good
- Van #2: Status=Ready, Condition=Good
- Van #3: Status=Under Maintenance (ticket for engine repair), Condition=Needs Maintenance
- Van #4: Status=Under Maintenance (ticket for brake work), Condition=Needs Maintenance

Dashboard Alert: Fragility Detection does NOT trigger (2 ready vans is safe). System shows: "Transport Vans: 2/4 Ready, 2/4 Under Maintenance".

Risk Scenario: Van #1 breaks down unexpectedly (engine failure). Immediately:

- Van #1 Status=Under Maintenance, only 1 ready van remains (Van #2)
- Dashboard shows FRAGILE alert: "Only 1 ready Transport Van. If this unit fails, zero vans available"
- Admin prioritizes finishing Van #3 or Van #4 repairs to restore redundancy

Resolution: Mechanic accelerates Van #3 repairs. Within 2 days, Van #3 is ready. Fragile alert clears. 2 vans are ready again.

Key Insight: Fragility Detection doesn't prevent simultaneous maintenance (it's sometimes necessary), but it alerts Admins to single-point-of-failure risk and triggers corrective action.

Scenario 7: Custodian Rejects Repair for Not Meeting Specifications

Setup: Ticket #88 is to rebuild the transmission to "OEM specifications" (Original Equipment Manufacturer). Mechanic completes work. Sub-issue: "Rebuild transmission" is "For Inspection".

Custodian A Action: Inspects the rebuild, test-drives, listens for smooth shifts. Finds that shift timing is slightly off (0.5 second delay between gears—acceptable for normal use, but not OEM spec). Custodian marks inspection "Rejected".

Rejection Note: "Shift timing out of spec. OEM spec requires <0.2 second delay; actual is ~0.5 second. Requires recalibration."

System Response: Sub-issue returns to "For Inspection" (awaiting rework).

Mechanic A Action: Receives rejection notification, reviews the specification, recalibrates transmission. Marks sub-issue "For Inspection" again.

Custodian A Action (Retry): Re-tests, confirms shift timing is now <0.2 second. Marks inspection "Approved".

Admin Confirmation: Reviews Custodian's approval, confirms the work meets specs, marks sub-issue "Done".

Key Insight: The inspection gate (Custodian) enforces adherence to specifications. Without it, Mechanic could "good enough" the fix (functional but not compliant), and the customer would receive substandard work.

Scenario 8: False Alarm Issue Report (No Ticket Created)

Setup: During Readiness Check, Custodian B notices the water temperature gauge shows "high" (180°F). Creates Issue Report: "Engine temperature abnormally high".

Admin Review: Reads the Issue Report. Checks the vehicle's temperature logs and finds:

- Gauge readings for the past week consistently around 170-180°F
- This is actually normal for the vehicle model under load (not an anomaly)
- The gauge is functioning correctly; there's no overheating

Admin Decision: "This is a false alarm. The temperature is normal. No ticket needed. I'll close this Issue Report as 'Monitored—No Action Required'".

System Actions: Issue Report is closed. No Maintenance Ticket is created. Vehicle Status remains unchanged.

Future: If temperature readings exceed 190°F, a new Issue Report would be created and a ticket would follow.

Key Insight: The system separates Issue Reports (low-cost to create) from Maintenance Tickets (high-cost, triggers work). Not every issue becomes a ticket. This allows documentation of observations without creating unnecessary work.

Scenario 9: Preventive Maintenance Completed and Auto-Recurs

Setup: Admin A created a Preventive Maintenance Schedule on Jan 1:

- Vehicle #5: "Oil and Filter Change"
- Frequency: Every 6 months
- First occurrence: Jan 1, 2026

Jan 1, 2026: System auto-generates Ticket #200: "Preventive Oil Change - Vehicle #5".

Jan 5: Mechanic C is assigned, performs oil change, marks Ticket #200 as "Done".

System Response: Upon closing, system auto-generates the next occurrence: Ticket #301 scheduled for July 1, 2026.

July 1, 2026: Ticket #301 is auto-created. Mechanic D is assigned, completes the work. Upon closing, next occurrence is auto-generated: Ticket #402 for Jan 1, 2027.

Skipped Occurrence: If Jan 1 passes and Ticket #200 is still open (not closed), no new ticket is created on July 1. The system does NOT skip to the next date—it waits for the current ticket to close before generating the next one.

Key Insight: Auto-recurrence ensures preventive maintenance never gets "forgotten". The schedule is self-perpetuating as long as mechanics complete the work on time.

Scenario 10: Vehicle Decommissioned Then Later Restored

Setup: Vehicle #15 (old ambulance) was decommissioned in June 2025 (Status=Decommissioned) due to age and high maintenance costs.

Change of Circumstances: In June 2026, a newer ambulance breaks down (major repairs needed). Organization buys back Vehicle #15 from the salvage lot (it was not scrapped, just parked).

Admin Options:

1. **Option A:** Admin changes Vehicle #15 Status from "Decommissioned" to "Out of Service" (restoration attempt)
2. **Option B:** Create a new vehicle record (history remains clean, different tracking)

Option A Workflow (If chosen):

- Admin manually changes Status to "Out of Service"
- Custodian performs full Condition Check (vehicle hasn't been inspected in a year—needs thorough assessment)
- If Condition = "Good", Custodian performs Readiness Check
- If both pass, Status can change to "Ready"
- History shows: "Decommissioned June 2025, Restored June 2026"

Option B Workflow (If chosen):

- Create new vehicle record: "Ambulance #16 (refurbished)"
- Old Vehicle #15 remains in history (read-only) for audit purposes
- New vehicle starts with fresh intake workflow

Recommendation: Option A if the vehicle was only parked (good condition expected). Option B if the vehicle was heavily used, sold, or modified (different vehicle effectively).

Key Insight: Decommissioning is not a one-way door, but restoration requires fresh verification (Condition/Readiness checks). The system doesn't assume a restored vehicle is fit for service; it requires proving it.

Section 6: Technical Implementation Questions (10 Q&A)

Q6.1: How are Notifications triggered in the VMS?

Event-Driven Architecture:

- Backend (Laravel) monitors for key state changes
- When a state change occurs (ticket assigned, sub-issue for inspection, etc.), backend triggers a notification
- Frontend polls or subscribes to notification queue, displays in-app notification

Notification Events:

- Ticket Assigned → "You've been assigned Ticket #42"
- Sub-Issue For Inspection → "Sub-issue #1 ready for your inspection"
- Sub-Issue Approval/Rejection Result → "Your repair was [Approved/Rejected]"
- Ticket Closed → "Ticket #42 is now closed"
- Preventive Maintenance Due → "Preventive maintenance for Vehicle #5 due today"

Storage: Notifications are stored in the database (table: notifications) with read/unread flags.

Delivery: Currently in-app only. Future: Email notifications as enhancement.

Q6.2: How does the Dashboard calculate coverage percentage?

Coverage % Formula: (Vehicles with current Condition Check) / (Total Vehicles) × 100%

What is "current"?: A Condition Check performed within the past 12 months (or a configurable threshold).

Calculation Details:

- Query database: SELECT COUNT(*) FROM vehicles WHERE (most recent condition_check).date >= NOW() - INTERVAL 12 MONTHS
- Divide by total active vehicles (exclude decommissioned)
- Round to nearest integer percentage

Dashboard Display: "Coverage: 85%" → "17 of 20 vehicles have current Condition Checks"

Alert: If coverage drops below a threshold (e.g., 75%), dashboard shows yellow/red alert: "Low Coverage. Some vehicles are overdue for Condition Checks."

Q6.3: How are Permission Checks enforced at the API level?

Backend Middleware (Laravel):

- Every API endpoint has a permission check before executing logic
- Example endpoint: POST /tickets/{id}/verify-main-issue (Verify Main Issue)
- Permission check: `if (auth()->user()->role !== 'Custodian' && auth()->user()->role !== 'Admin') { return 403 Forbidden; }`

Common Permission Patterns:

- Verify Main Issue → Only Custodian or Admin
- Confirm Sub-Issue → Only Admin
- Close Ticket → Only Admin
- Update Sub-Issue Progress → Only assigned Mechanic
- View Ticket → Assigned Custodian, assigned Mechanic, or Admin

Error Response: If permission denied, API returns 403 with message: "You don't have permission to perform this action".

Audit Trail: All permission checks are logged for compliance.

Q6.4: How are Maintenance Records created and stored?

Creation Trigger: When a Maintenance Ticket is closed (Done or Decision Close), a Maintenance Record is auto-created.

Data Captured:

- Ticket ID (reference to original ticket)
- Vehicle ID
- Description of work performed
- Mechanic(s) involved
- Date range (started, completed)
- Sub-issues completed (list of what was fixed)
- Cost (if tracked)
- Result (Done vs. Decision Close)
- Deferred work (if any)

Immutability: Records cannot be edited or deleted once created. Only Admins can archive old records.

Storage: Maintenance Records table (table: maintenance_records) in database.

Accessibility: Admins can view, search, and export records. Custodians can view records for their vehicles. Mechanics see only records they worked on.

Q6.5: How does deferral automatically create Issue Report breadcrumbs?

Deferral Logic (Backend):

1. Mechanic marks Sub-Issue status = "Deferred" with note: "Engine replacement—backordered"
2. Backend detects sub-issue state change to Deferred
3. Trigger: Auto-create Issue Report with:
    - Vehicle ID (from ticket)
    - Description: "Deferred work from Ticket #XXX: Engine replacement—backordered"
    - Status: "Requires Follow-up"
    - Priority: Inherit from ticket (if urgent)
4. New Issue Report appears in Admin's Issues queue

Later Workflow:

- Admin reviews Issue Report and decides: (a) create new ticket, (b) monitor it, or (c) close it
- If new ticket is created, it references the Issue Report for continuity

Prevention: Without this auto-generation, deferred work would disappear once the ticket closed. The automatic breadcrumb ensures nothing is lost.

Q6.6: How are Vehicle Histories preserved and made immutable?

History Table: A separate audit log table (table: vehicle_histories) records every state change.

Events Logged:

- Status change: Ready → Under Maintenance
- Condition change: Good → Needs Maintenance
- Location change: Central Station → North Depot
- Check performed: Readiness Check result = Ready/Not Ready
- Decommissioning/Restoration

Immutability: History records are INSERT-only (write once, never update or delete). This creates an append-only log.

Querying History: Frontend queries the history table and displays in reverse chronological order (newest first).

Compliance: Complete audit trail shows "what changed, when, and who did it". Useful for compliance audits, debugging, and forensics.

Q6.7: How does the React SPA handle state management across components?

State Management Pattern: React hooks (useState, useContext) manage local and shared state.

Component Hierarchy:

- **App.jsx:** Root component. Manages user auth state, theme preference, global UI state
- **Workspace.jsx:** Main dashboard. Manages vehicles list, selected vehicle, active view
- **SmartForm.jsx:** Form component. Manages form state (fields, errors, submission state)
- **TableComponent.jsx:** Table display. Manages sorting, filtering, pagination

API Integration:

- Components fetch data from backend via REST API (GET, POST, PUT, DELETE)
- Data is stored in React state, not duplicated
- Changes trigger re-fetches to keep state in sync

Example: User submits a form. SmartForm sends POST request → Backend validates and saves → Backend returns success. SmartForm updates state, closes overlay, shows success toast. Dashboard re-fetches vehicle list and updates display.

Q6.8: How does the backend enforce business logic constraints?

Validation Layers:

- **Database Constraints:** Foreign keys, unique constraints, check constraints enforce data integrity at DB level
- **Model Validation (Laravel):** Eloquent models define validation rules (required, unique, date_after, etc.)
- **Controller Validation:** Request validation happens in controllers before processing logic
- **Business Logic Rules:** Custom methods enforce rules (e.g., "Cannot close ticket if sub-issues are not Done")

Examples:

- **State Transitions:** A method `transitionTo($state)` checks if transition is valid before allowing it
- **Permission Checks:** Before updating a ticket, verify user role has permission
- **Data Integrity:** Before deleting a vehicle, check if it has related tickets. If yes, prevent deletion

Error Responses: If validation fails, API returns 422 Unprocessable Entity with field-level errors. Frontend displays these to user.

Q6.9: How is the database schema designed to support the VMS functionality?

Core Tables:

- **vehicles:** id, name, type_id, location_id, status, condition, created_at, updated_at
- **vehicle_types:** id, name (Ambulance, Fire Truck, etc.)
- **locations:** id, name, hub_name (Central Station, North Depot)
- **condition_checks:** id, vehicle_id, result (Good/Needs Maintenance/Not Fit), date, custodian_id
- **readiness_checks:** id, vehicle_id, result (Ready/Not Ready), reason, date, custodian_id, expires_at
- **issue_reports:** id, vehicle_id, description, status, created_by, date
- **maintenance_tickets:** id, vehicle_id, issue_report_id, status (Open/Active/Closed), mechanic_id, custodian_id
- **sub_issues:** id, ticket_id, description, status (Open/Under Repair/For Inspection/For Confirmation/Done/Deferred)
- **maintenance_records:** id, ticket_id, vehicle_id, description, closed_date (immutable)
- **maintenance_schedules:** id, vehicle_id, task_description, frequency_days, last_run_date
- **users:** id, name, email, role (Admin/Custodian/Maintenance)
- **vehicle_histories:** id, vehicle_id, event_type, old_value, new_value, timestamp (append-only log)

Relationships: Foreign keys link vehicles → conditions, tickets → vehicles, sub_issues → tickets, etc.

Indexes: On frequently queried columns (vehicle_id, status, created_at) for performance.

Q6.10: How does the system export data (e.g., for reporting or external audits)?

Export Functionality: Admins can export data as CSV from several views:

- **Vehicles Export:** All vehicle records (ID, name, type, status, condition, location, last check date)
- **Tickets Export:** Closed/archived tickets (ID, vehicle, mechanic, date range, result, cost if tracked)
- **Maintenance Records Export:** Work history (vehicle ID, work performed, mechanic, dates, deferred items)
- **Reports Export:** Readiness report, condition report, mechanic productivity report

Technical Implementation:

- Backend generates CSV file (Laravel uses Excel libraries or manual CSV generation)
- File is returned to frontend for browser download
- Files are not stored on disk (temporary in-memory generation)

Audit Uses:

- External auditors can review work history for compliance
- Finance teams can analyze maintenance costs by vehicle or mechanic
- Operations can trend readiness over time (% ready each month)
